

# Lab Assignment #11

Math 437 - Modern Data Analysis

## Instructions

In this lab we will look at decision trees and ensembles of decision trees for both classification and regression.

```
here::i_am("Labs/Lab11.qmd")

# You may not need all of this, but you'll need most of it
library(ISLR2)
library(dplyr)
library(rpart)
library(randomForest)
library(gbm)
library(tidymodels)
library(ranger)
library(vip)
library(xgboost)

# load and split public_2020 dataset
public <- read.csv("../Data Sets/public_2020.csv")
public <- public |>
  mutate(
    MSI = if_else(HBCU | HSI, "Yes", "No") |> as.factor()
  )

set.seed(3745)
public_split <- initial_split(
  public,
  prop = 0.75
)
public_train <- training(public_split)
public_test <- testing(public_split)
```

This lab assignment is worth a total of **25 points** with up to 5 points of extra credit.

## Problem 1: Decision Trees

In past years, students have complained that the **tree** package wasn't running on their version of R. So instead of doing Labs 8.3.1 and 8.3.2 we will run similar code using the **rpart** package.

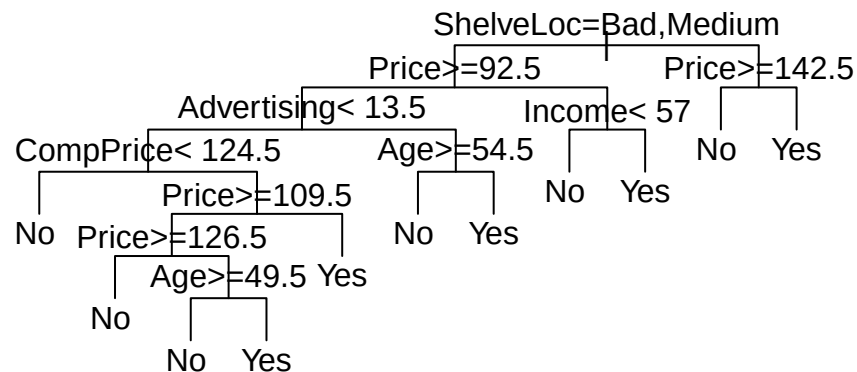
I have written each of the code chunks below to match Labs 8.3.1 and 8.3.2 as closely as possible. For each part, run the corresponding code chunks and answer the question about the output. You may find reading the text in Labs 8.3.1 and 8.3.2 to be useful.

### Part a (Explanation: 1 pt)

```
Carseats2 <- Carseats |>
  mutate(
    High = if_else(Sales <= 8, "No", "Yes"))

tree.carseats <- rpart(High ~. - Sales, data = Carseats2)

par(xpd = TRUE)
plot(tree.carseats, uniform = TRUE)
text(tree.carseats, pretty = 0)
```



Briefly explain how to interpret this tree: which stores would end up in the very left “no” group, and which stores would end up in the very right “yes” group?

### Solution

The tree is interpreted by following the sequence of splits from the top to a terminal node. Stores in the far left “No” group have bad or medium shelf location, higher prices, low advertising, and lower competitor prices, leading to low sales. Stores in the far right “Yes” group tend to have lower prices, which leads to high sales despite shelf location. ### Part b (Explanation: 1 pt)

```
tree.carseats
```

```
n= 400
```

```
node), split, n, loss, yval, (yprob)
```

```
* denotes terminal node
```

```

1) root 400 164 No (0.59000000 0.41000000)
 2) ShelveLoc=Bad,Medium 315 98 No (0.68888889 0.31111111)
 4) Price>=92.5 269 66 No (0.75464684 0.24535316)
 8) Advertising< 13.5 224 41 No (0.81696429 0.18303571)
16) CompPrice< 124.5 96 6 No (0.93750000 0.06250000) *
```

```

17) CompPrice>=124.5 128 35 No (0.72656250 0.27343750)
34) Price>=109.5 107 20 No (0.81308411 0.18691589)
68) Price>=126.5 65 6 No (0.90769231 0.09230769) *
69) Price< 126.5 42 14 No (0.66666667 0.33333333)
138) Age>=49.5 22 2 No (0.90909091 0.09090909) *
139) Age< 49.5 20 8 Yes (0.40000000 0.60000000) *
35) Price< 109.5 21 6 Yes (0.28571429 0.71428571) *
9) Advertising>=13.5 45 20 Yes (0.44444444 0.55555556)
18) Age>=54.5 20 5 No (0.75000000 0.25000000) *
19) Age< 54.5 25 5 Yes (0.20000000 0.80000000) *
5) Price< 92.5 46 14 Yes (0.30434783 0.69565217)
10) Income< 57 10 3 No (0.70000000 0.30000000) *
11) Income>=57 36 7 Yes (0.19444444 0.80555556) *
3) ShelveLoc=Good 85 19 Yes (0.22352941 0.77647059)
6) Price>=142.5 12 3 No (0.75000000 0.25000000) *
7) Price< 142.5 73 10 Yes (0.13698630 0.86301370) *

```

What do the numbers in the parentheses mean? What do the stars mean?

### Solution

The numbers in parentheses represent the estimated class probabilities at each node. For example, (0.59, 0.41) means a 59% probability of “No” and a 41% probability of “Yes.” The stars (\*) indicate terminal nodes, where the tree stops splitting and a final classification is made. ### Part c (Explanation: 0.5 pts)

```

set.seed(2)
train <- sample(1:nrow(Carseats2), 200)
Carseats.test <- Carseats2[-train,]

tree.carseats <- rpart(High ~ . - Sales,
                        data = Carseats2,
                        subset = train,
                        control = rpart.control(xval = 10))
names(tree.carseats)

```

|                            |           |             |
|----------------------------|-----------|-------------|
| [1] "frame"                | "where"   | "call"      |
| [4] "terms"                | "cptable" | "method"    |
| [7] "parms"                | "control" | "functions" |
| [10] "numresp"             | "splits"  | "csplit"    |
| [13] "variable.importance" | "y"       | "ordered"   |

```
tree.pred <- predict(tree.carseats, Carseats.test, type = "class")
table(tree.pred, Carseats.test$High, dnn = c("Predicted", "Actual"))
```

|           | Actual |     |
|-----------|--------|-----|
| Predicted | No     | Yes |
| No        | 97     | 25  |
| Yes       | 20     | 58  |

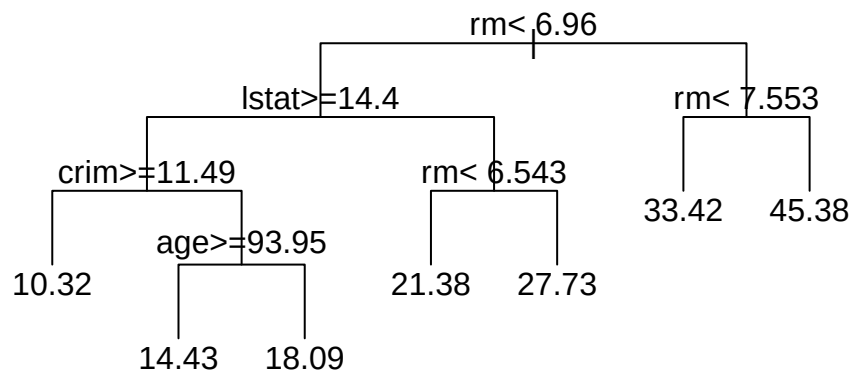
What does the argument `xval = 10` mean?

### Solution

The argument `xval = 10` indicates that 10-fold cross-validation is used to evaluate the model. The data is divided into 10 subsets, and the model is trained and validated 10 times, each time leaving out one subset. This helps estimate the model's prediction error and select the optimal tree size. ### Part d (Explanation: 1 pt)

```
set.seed(1)
n <- nrow(Boston)
train <- sample(1:n, n/2)
tree.boston <- rpart(medv~., Boston, subset = train, control = rpart.control(xval = 10))
```

```
par(xpd = TRUE)
plot(tree.boston, uniform = TRUE)
text(tree.boston, pretty = 0)
```



Briefly explain how to interpret this tree: which neighborhoods would be predicted to have a `medv` of 10.32? Which neighborhoods would be predicted to have a `medv` of 27.73?

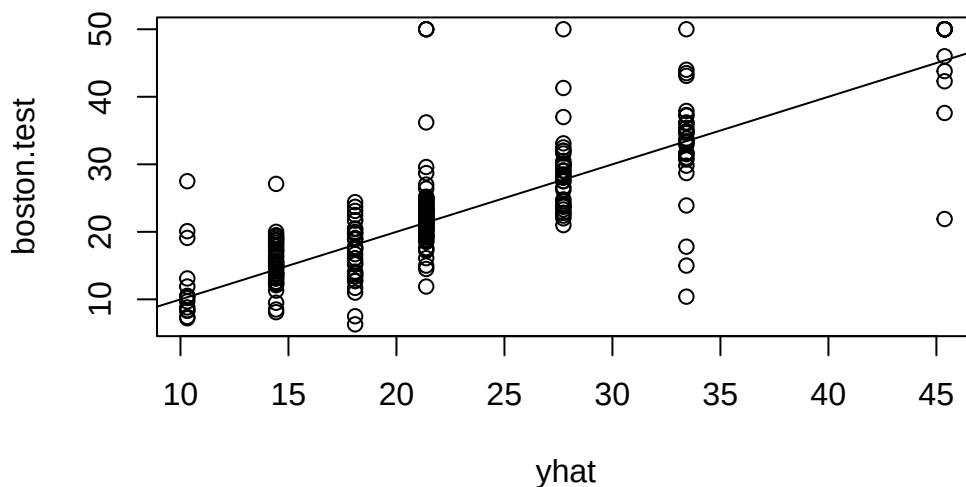
### Solution

The tree is interpreted by following the sequence of splits to a terminal node. Neighborhoods with `rm < 6.96`, `lstat < 14.4`, and `crim < 11.49` are predicted to have a median home value of 10.32. Neighborhoods with `rm < 6.96`, `lstat < 14.4`, and `rm < 6.543` are predicted to have a median home value of 27.73. ### Part e (Explanation: 1 pt)

```

yhat <- predict(tree.boston, Boston[-train,])
boston.test <- Boston$medv[-train]
plot(yhat, boston.test)
abline(0,1)

```



```
mean((yhat - boston.test)^2)
```

```
[1] 35.28688
```

Why do we get this pattern - a bunch of vertical lines - in the plot? (HINT: We saw plots like this in the example code from lecture, but they were horizontal lines because the predictions were on the vertical axis instead.)

### Solution

The vertical lines occur because a regression tree makes piecewise constant predictions. This means many observations receive the same predicted value, so multiple points share the same x-value (yhat), forming vertical lines in the plot. If the predictions were plotted on the y-axis instead, the lines would appear horizontal. ### Part f (Code: 2.5 pts; Explanation: 1.5 pts)

We are now going to predict which public institutions are minority-serving institutions (HBCU or HSI).

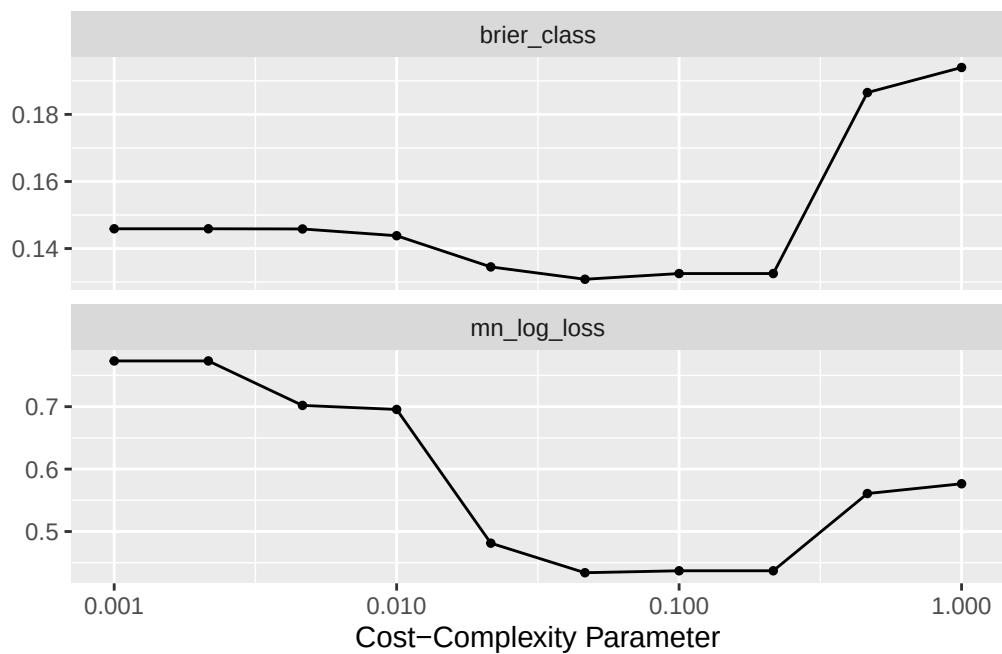
Using `tidymodels` with the `rpart` package, fit a classification tree model on the training set. Tune the cost-complexity parameter using a grid of values between 0.01 and 1 with a reasonable enough number of levels so that you can actually see how your error metric of choice changes as a function of the cost-complexity parameter.

Choose a cost-complexity parameter by either Brier score or mean log-loss and fit that “best” model on the training set. Plot the final classification tree and explain how to use the tree to predict whether a school is MSI vs. non-MSI.

## Solution

```
treeC_model <- decision_tree(  
  mode = "classification",  
  engine = "rpart",  
  cost_complexity = tune()  
)  
  
treeC_recipe <- recipe(  
  MSI ~ Admission_Rate + SAT + Undergraduates + Average_Total_Cost +  
    Instructional_Expenditure + Faculty_Salary + Full_Time_Faculty +  
    Pell_Grant + Retention_Rate + Federal_Loan + Default_Rate +  
    Proportion_Men + Endowment + Location,,  
  data = public_train  
)  
  
treeC_wflow <- workflow() |>  
  add_model(treeC_model) |>  
  add_recipe(treeC_recipe)  
  
set.seed(3745)  
  
public_kfold <- vfold_cv(  
  public_train,  
  v = 5,  
  repeats = 3,  
  strata = MSI  
)  
  
treeC_tune <- tune_grid(  
  treeC_wflow,  
  resamples = public_kfold,  
  grid = grid_regular(  
    cost_complexity(range = c(-3, 0)),  
    levels = 10  
  ),  
  metrics = metric_set(brier_class, mn_log_loss)  
)
```

```
autoplot(treeC_tune)
```



```
#| label: choose best classification tree
```

```
treeC_best <- select_best(  
  treeC_tune,  
  metric = "brier_class"  
)
```

```
treeC_best
```

```
# A tibble: 1 x 2  
  cost_complexity .config  
    <dbl> <chr>  
1    0.0464 Preprocessor1_Model106
```

```
#| label: fit final classification tree
```

```
treeC_wflow_final <- finalize_workflow(  
  treeC_wflow,  
  parameters = treeC_best
```

```
)

treeC_fit <- fit(
  treeC_wflow_final,
  data = public_train
)

treeC_fit
```

```
== Workflow [trained] =====
Preprocessor: Recipe
Model: decision_tree()

-- Preprocessor -----
0 Recipe Steps

-- Model -----
n= 300

node), split, n, loss, yval, (yprob)
      * denotes terminal node

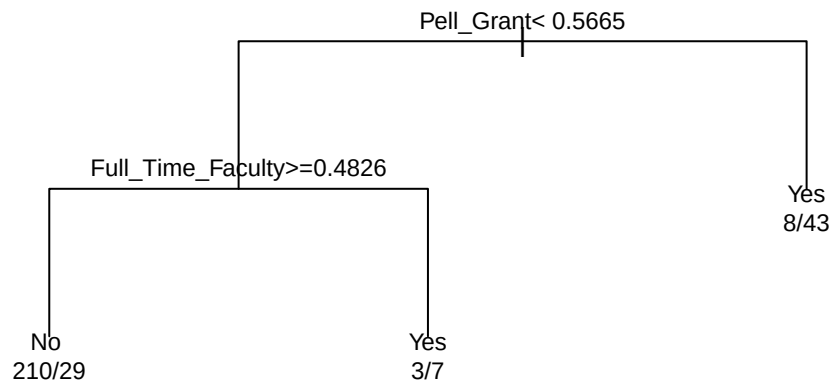
1) root 300 79 No (0.7366667 0.2633333)
  2) Pell_Grant< 0.56645 249 36 No (0.8554217 0.1445783)
    4) Full_Time_Faculty>=0.4826 239 29 No (0.8786611 0.1213389) *
    5) Full_Time_Faculty< 0.4826 10 3 Yes (0.3000000 0.7000000) *
  3) Pell_Grant>=0.56645 51 8 Yes (0.1568627 0.8431373) *
```

```
#| label: plot final MSI classification tree

par(xpd = TRUE)

treeC_fit |>
  extract_fit_engine() |>
  plot(uniform = TRUE)

treeC_fit |>
  extract_fit_engine() |>
  text(use.n = TRUE, minlength = 4, cex = 0.75)
```



Explanation: I tuned the cost-complexity parameter using cross-validation and selected the best model based on the Brier score. The tuning plot shows that model performance improves as cost-complexity increases up to a point, and then worsens, reflecting the bias-variance tradeoff. The optimal value was approximately 0.0464.

The final classification tree shows that the most important predictor is the proportion of students receiving Pell Grants. Schools with higher Pell Grant rates are more likely to be classified as MSI. For schools with lower Pell Grant rates, the proportion of full-time faculty further splits the data, with higher proportions associated with non-MSI institutions.

To predict whether a school is MSI, we follow the splits in the tree from the top to a terminal node. The final node assigns a classification based on the majority class in that region.

### Part g (Code: 1 pt)

Using your final model from Part f, make predictions on the test set. Obtain the confusion matrix and compute the estimated Brier score or mean log-loss.

### Solution

```
tree_preds <- predict(treeC_fit, new_data = public_test, type = "prob")
tree_class <- predict(treeC_fit, new_data = public_test, type = "class")
```

```

tree_results <- bind_cols(
  public_test,
  tree_class,
  tree_preds
)

# confusion matrix
conf_mat(
  data = tree_results,
  truth = MSI,
  estimate = .pred_class
)

```

|            | Truth |     |
|------------|-------|-----|
| Prediction | No    | Yes |
| No         | 72    | 15  |
| Yes        | 8     | 6   |

```

# Brier score
brier_class(
  data = tree_results,
  truth = MSI,
  .pred_Yes
)

```

```

# A tibble: 1 x 3
  .metric      .estimator .estimate
  <chr>        <chr>         <dbl>
1 brier_class binary          0.599

```

## Problem 2: Random Forests and Boosted Trees

### Part a (Code: 2 pts)

Run the code in ISLR Labs 8.3.3 and 8.3.4. Put each chunk from the textbook in its own chunk.

## Solution

Tired ### Part b (Code: 2.5 pts)

Using tidymodels, create a random forest model predicting whether an institution is a minority-serving institution (MSI) from the other variables in the dataset (except, obviously, HBCU and HSI, and Name). The `rand_forest` function will accept either `randomForest` or `ranger` as the engine.

Make sure to use `set_args` to tell tidymodels you want to use permutation-based importance (you will need it for part d). Tune the value of `mtry` using cross-validation; use values of `mtry` from 1 to 14 (14 = bagging).

Select your best model by cross-validated Brier score and justify your choice. Using your best model, obtain the out-of-bag prediction error (Brier score) on the training set.

## Solution

```
#Model
rf_model <- rand_forest(
  mode = "classification",
  mtry = tune(),
  trees = 500
) |>
  set_engine("ranger") |>
  set_args(
    mtry = tune(),
    importance = "permutation"
  )

#Recipe
rf_recipe <- recipe(
  MSI ~ .,
  data = public_train
) |>
  step_rm(Name, HBCU, HSI)

#Workflow
rf_wflow <- workflow() |>
  add_model(rf_model) |>
  add_recipe(rf_recipe)

#CV
set.seed(3745)
```

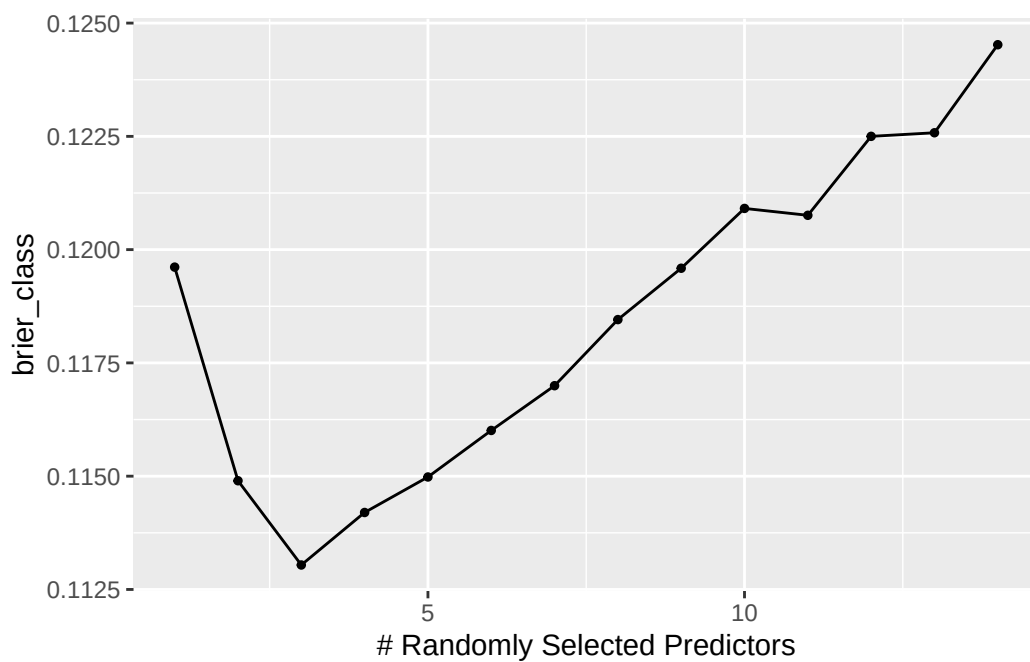
```

rf_folds <- vfold_cv(
  public_train,
  v = 5,
  repeats = 3,
  strata = MSI
)

rf_grid <- tibble(mtry = 1:14)

rf_tune <- tune_grid(
  rf_wflow,
  resamples = rf_folds,
  grid = rf_grid,
  metrics = metric_set(brier_class)
)
#Plot tuning
autoplot(rf_tune)

```



```

#best model
rf_best <- select_best(
  rf_tune,
  metric = "brier_class"
)

```

```

)

rf_best

# A tibble: 1 x 2
  mtry .config
  <int> <chr>
1     3 Preprocessor1_Model03

#Final model
rf_final <- finalize_workflow(
  rf_wflow,
  rf_best
)

rf_fit <- fit(
  rf_final,
  data = public_train
)

#OOB Brier score
rf_fit_engine <- extract_fit_engine(rf_fit)

rf_fit_engine$prediction.error

[1] 0.1127798

```

Explanation: The best random forest model selected by cross-validation used `mtry=3`. The out-of-bag Brier score for the final model was approximately 0.113. Since lower Brier scores indicate better probability predictions, this suggests the random forest model performs fairly well on the training data using OOB error estimation.

### Part c (Code: 1 pt; Explanation: 0.5 pts)

Using the model chosen in Part b, obtain predictions on the test set and compute the estimated test Brier score. Compare Brier score on the test set to the cross-validated mean Brier score and out-of-bag prediction error on the training set.

### Solution

```
# test predictions
rf_test_probs <- predict(
  rf_fit,
  new_data = public_test,
  type = "prob"
)

# Brier score on test set
rf_test_brier <- brier_class_vec(
  truth = public_test$MSI,
  estimate = rf_test_probs$.pred_Yes
)

rf_test_brier
```

```
[1] 0.648517
```

```
rf_cv_brier <- rf_tune |>
  collect_metrics() |>
  filter(
    .metric == "brier_class",
    mtry == rf_best$mtry
  ) |>
  pull(mean)

rf_cv_brier
```

```
[1] 0.11304
```

Explanation: The test set Brier score is approximately 0.649, while the cross-validated Brier score is about 0.113 and the out-of-bag Brier score is about 0.113 as well. Since lower Brier scores indicate better probability predictions, the test set performance is much worse than both the cross-validated and out-of-bag estimates. This suggests that the model does not generalize well to new data and may be overfitting the training set or is sensitive to differences between the training and test sets.

#### Part d (Code: 0.5 pts; Explanation: 1 pt)

Obtain and plot estimates of variable importance (%IncMSE column if you are using `randomForest` or `variable.importance` if you are using `ranger`) for the model chosen in

Part b. Which predictor variables have the largest importance and are thus the most useful for making the predictions?

### Solution

```
# extract fitted ranger model
rf_fit_engine <- extract_fit_engine(rf_fit)

# get variable importance
rf_importance <- rf_fit_engine$variable.importance

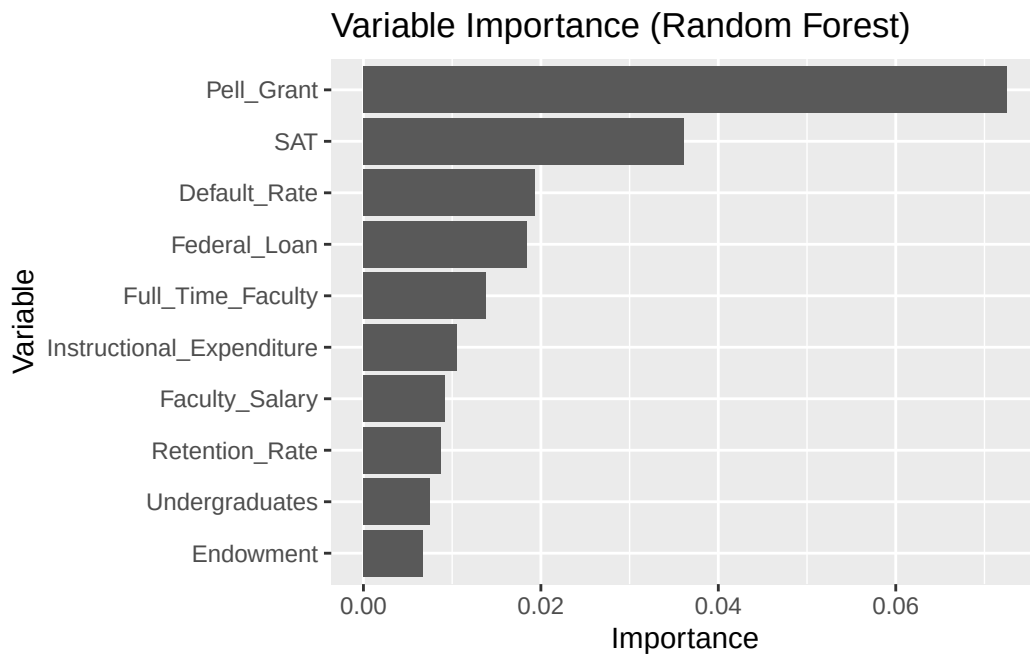
# convert to tibble and sort
rf_importance_tbl <- tibble(
  variable = names(rf_importance),
  importance = rf_importance
) |>
  arrange(desc(importance))

rf_importance_tbl
```

```
# A tibble: 14 x 2
  variable      importance
  <chr>         <dbl>
1 Pell_Grant    0.0725
2 SAT           0.0361
3 Default_Rate  0.0193
4 Federal_Loan  0.0185
5 Full_Time_Faculty 0.0138
6 Instructional_Expenditure 0.0105
7 Faculty_Salary 0.00922
8 Retention_Rate 0.00870
9 Undergraduates 0.00749
10 Endowment    0.00675
11 Average_Total_Cost 0.00660
12 Admission_Rate 0.00259
13 Proportion_Men 0.00236
14 Location     0.000282
```

```
rf_importance_tbl |>
  slice_max(importance, n = 10) |>
  ggplot(mapping = aes(x = reorder(variable, importance), y = importance)) +
```

```
geom_col() +
coord_flip() +
labs(
  title = "Variable Importance (Random Forest)",
  x = "Variable",
  y = "Importance"
)
```



Explanation: The variable importance plot shows that the most important predictor in the random forest model is Pell\_Grant, by a large margin. This indicates that the proportion of students receiving Pell Grants is the most influential factor in determining MSI status. Other important variables include SAT, Default\_Rate, and Federal\_Loan, though their importance is much smaller in comparison. Overall, this suggests that socioeconomic factors play a key role in predicting whether an institution is classified as an MSI.

### Part e (Code: 2.5 pts)

Using tidymodels with the `xgboost` package, fit a gradient-boosted tree model on the training set using the same predictors as the decision tree and random forest models. Tune only the 3 hyperparameters discussed in lecture: the number of trees to fit (`trees`), interaction depth (`tree_depth`), and learning rate (`learn_rate`). In order for this to not take forever, search

over a small grid: 50, 100, 250, and 500 for `trees`, 1 and 2 for `tree_depth`, and 0.01 and 0.10 for `learn_rate`.

Select your best model by either Brier score or mean log-loss and justify your choice.

### Solution

```
# boosted tree model using xgboost
boost_model <- boost_tree(
  mode = "classification",
  engine = "xgboost",
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune()
)

# recipe
boost_recipe <- recipe(
  MSI ~ .,
  data = public_train
) |>
  step_rm(Name, HBCU, HSI) |>
  step_dummy(all_nominal_predictors())

# workflow
boost_wflow <- workflow() |>
  add_model(boost_model) |>
  add_recipe(boost_recipe)
#| label: boost tree tuning grid

# grid of hyperparameter values
boost_grid <- expand_grid(
  trees = c(50, 100, 250, 500),
  tree_depth = c(1, 2),
  learn_rate = c(0.01, 0.10)
)

#tune boosted tree model

set.seed(3745)

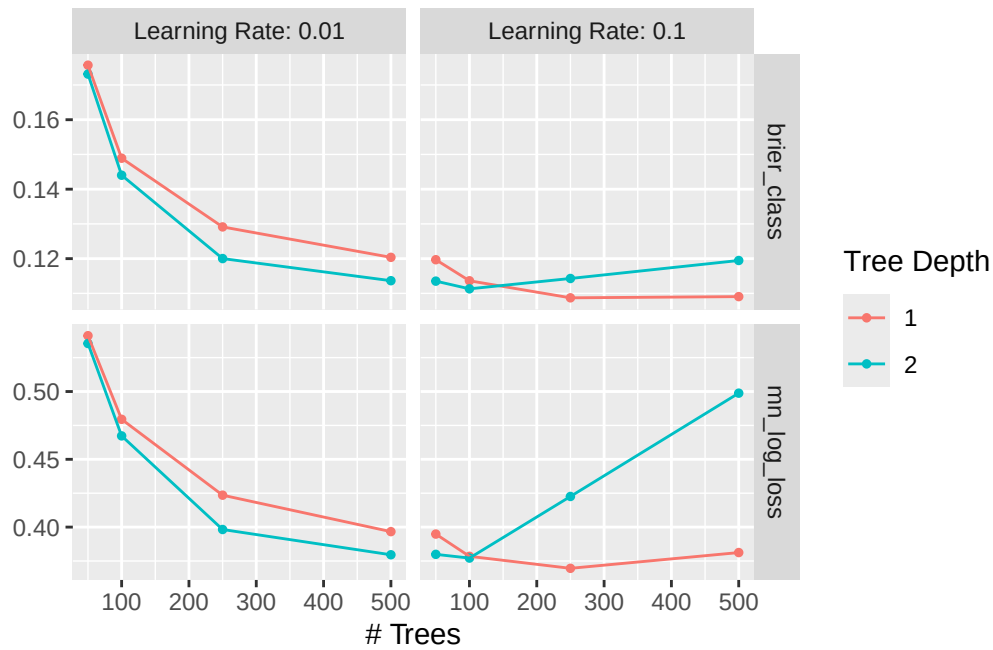
boost_tune <- tune_grid(
```

```

boost_wflow,
resamples = rf_folds,
grid = boost_grid,
metrics = metric_set(brier_class, mn_log_loss)
)

autoplot(boost_tune)

```



```

boost_best <- select_best(
  boost_tune,
  metric = "brier_class"
)

```

```
boost_best
```

```

# A tibble: 1 x 4
  trees tree_depth learn_rate .config
  <dbl>     <dbl>     <dbl> <chr>
1   250         1         0.1 Preprocessor1_Model107

```

```
#fit final boosted tree model

boost_final <- finalize_workflow(
  boost_wflow,
  parameters = boost_best
)

boost_fit <- fit(
  boost_final,
  data = public_train
)
```

### Part f (Code: 1 pt)

Using your best model from Part e, make predictions on the test set. Obtain the confusion matrix and the estimated Brier score or mean log-loss on the test set.

### Solution

```
# predicted probabilities
boost_test_probs <- predict(
  boost_fit,
  new_data = public_test,
  type = "prob"
)

# predicted classes
boost_test_class <- predict(
  boost_fit,
  new_data = public_test,
  type = "class"
)

# combine results
boost_results <- bind_cols(
  public_test,
  boost_test_class,
  boost_test_probs
)

#| label: confusion matrix for boosted tree
```

```
conf_mat(
  data = boost_results,
  truth = MSI,
  estimate = .pred_class
)
```

|            | Truth |     |
|------------|-------|-----|
| Prediction | No    | Yes |
| No         | 76    | 13  |
| Yes        | 4     | 8   |

```
#| label: test set Brier score for boosted tree
```

```
brier_class(
  data = boost_results,
  truth = MSI,
  .pred_Yes
)
```

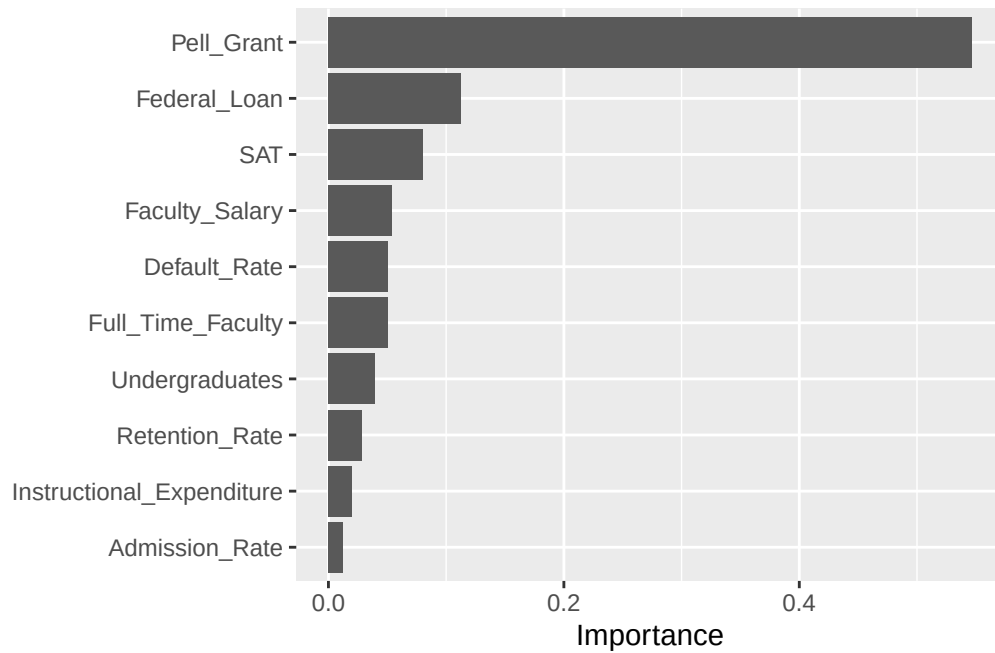
```
# A tibble: 1 x 3
  .metric      .estimator .estimate
  <chr>        <chr>      <dbl>
1 brier_class binary      0.678
```

### Part g (Code: 0.5 pts; Explanation: 1 pt)

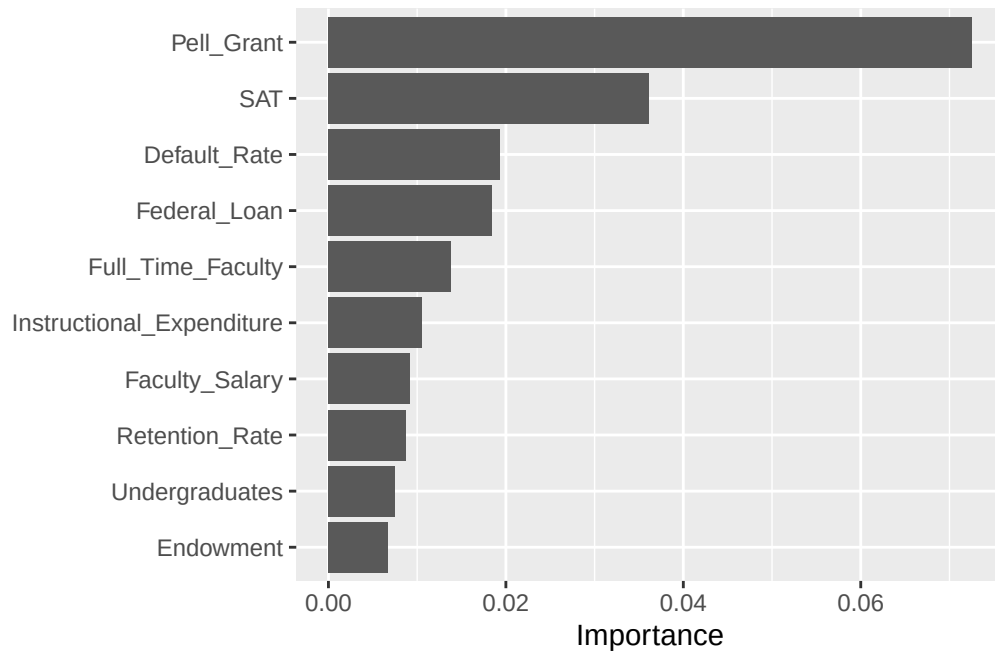
Produce a variable importance plot for your xgboost model from Part e. Compare the (relative) variable importances between the random forest and xgboost models.

### Solution

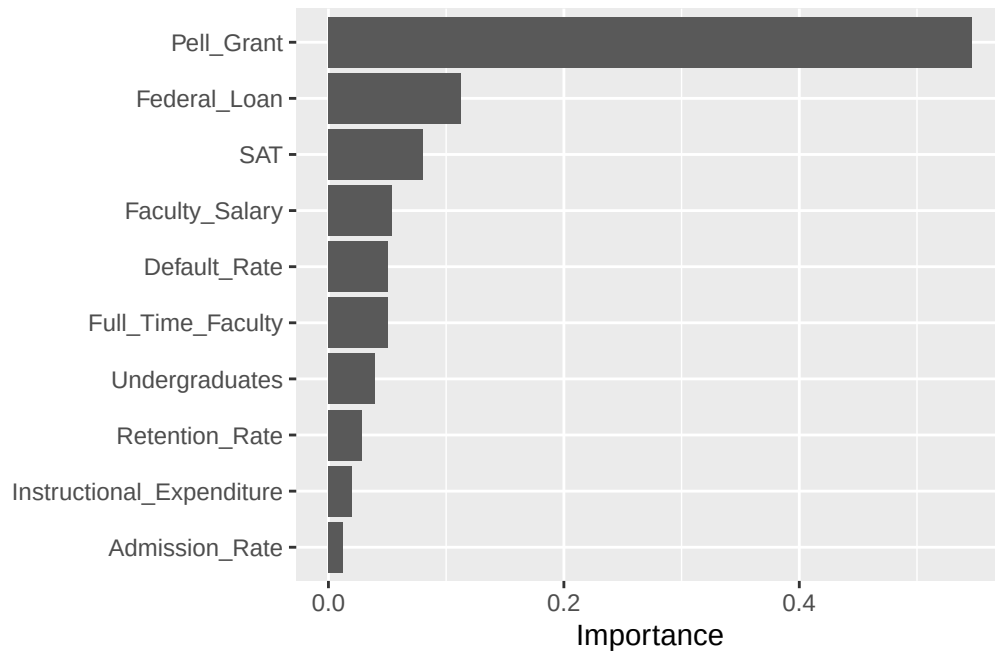
```
# Plot variable importance for the xgboost model
boost_fit |>
  extract_fit_parsnip() |>
  vip(num_features = 10)
```



```
#| label: compare RF and xgboost importance  
  
# Random forest variable importance plot  
rf_fit |>  
  extract_fit_parsnip() |>  
  vip(num_features = 10)
```



```
# Boosted tree variable importance plot
boost_fit |>
  extract_fit_parsnip() |>
  vip(num_features = 10)
```



Explanation: The variable importance plots for both the random forest and xgboost models show that Pell\_Grant is by far the most important predictor in both models. This suggests that the proportion of students receiving Pell Grants is the strongest factor in determining MSI status.

In both models, variables such as SAT, Federal\_Loan, and Default\_Rate also appear among the most important predictors, although their importance is much smaller compared to Pell\_Grant.

While the exact rankings differ slightly between the two models, the overall pattern is very similar, indicating that both models identify the same key variables as most influential. This consistency suggests that these variables are reliable predictors of MSI status.

#### Part h (Explanation: 1 pt)

Consider the following statement: the second decision tree that is fit does not depend on the first decision tree that is fit. Is that statement TRUE or FALSE for the random forest algorithm? What about for boosted trees? Explain your reasoning.

#### Solution

The statement is true for random forests but false for boosted trees. In a random forest, each tree is built independently using a random subset of the data and predictors, so the second

tree does not depend on the first tree at all. However, for boosted trees, the trees are built sequentially, where each new tree is trained to correct the errors made by the previous trees. This means the second tree does depend on the first tree in boosting.

### Problem 3: Neural Networks

#### Part a (Computation and Explanation: 2 pts)

Consider two predictors,  $x_1$  and  $x_2$ , feeding into a two-unit hidden layer. Suppose that the hidden layer uses ReLU activation functions with  $w_{kj}$  given in Equation (10.6) of the book and the output layer uses a linear activation function with  $\beta_j$  given in equation (10.6).

Find the activation functions  $A_1(x_1, x_2)$  and  $A_2(x_1, x_2)$  and  $f(x_1, x_2)$ . Note:  $f(x_1, x_2)$  should be a piecewise function.

#### Solution

The first hidden unit is

$$A_1(x_1, x_2) = \max(0, w_{11}x_1 + w_{21}x_2 + b_1)$$

The second hidden unit is

$$A_2(x_1, x_2) = \max(0, w_{12}x_1 + w_{22}x_2 + b_2)$$

The output layer is linear, so

$$f(x_1, x_2) = \beta_0 + \beta_1 A_1(x_1, x_2) + \beta_2 A_2(x_1, x_2)$$

Putting it together,

$$f(x_1, x_2) = \beta_0 + \beta_1 \max(0, w_{11}x_1 + w_{21}x_2 + b_1) + \beta_2 \max(0, w_{12}x_1 + w_{22}x_2 + b_2)$$

Because each ReLU is either 0 or its linear input, this function is piecewise linear.

#### Part b (Computation: 5 pts Extra Credit)

Run all code in the Chapter 10 Lab (ISLR, Sections 10.9.1-10.9.6). There are three options for running this code:

1. The default code in the textbook requires the `keras` package as well as a Python installation recognized by RStudio. The first time you try to set up `keras`, you will have to run `install_keras()` to actually install `keras`, `Tensorflow`, and their dependencies. If you do not have Python with Anaconda (or Miniconda) installed already on your device, you may want to follow the instructions in the error messages. If you cannot interpret an error message, please ask me.

- You probably cannot run GPU-based `keras` on your machine and will get a bunch of error messages when you first try to do anything with it. I was able to run the whole lab with CPU-based `keras`.
  - In Lab 10.9.2, `predict_classes()` is deprecated and may throw an error. I was able to get around the issue by replacing `predict_classes(x_test)` with `predict(x_test) %>% k_argmax() %>% as.numeric()`.
2. You can run the lab with the `torch` package using the code found at [the authors' website](#).
  - Past students have indicated that this code runs on their machine but is can sometimes be slow.
  3. You can run Lab 10 in Python using the [Jupyter notebook](#) accompanying *Introduction to Statistical Learning with Python*.

To keep this file from taking forever to render, open a *new* `.qmd` or `.ipynb` file, copy and run all code, and submit an HTML or pdf copy of the code and output (along with the original `.qmd` or `.ipynb` file).

Note that for Lab 10.9.4 (Using Pretrained CNN Models), I have found that if I put the `book_images` folder (unzipped) as a subfolder of the directory the lab is in, then everything will work as intended. If you get a “Permission denied” error, then you probably need to change the `img_dir` or move the folder around.